0

(Task 1 of 11) In this tutorial, we will learn about `begin`.

`begin` lets you evaluate a sequence of expressions and returns the value of the last expression.

1

(Task 2 of 11) What is the result of running this program?

```
(defvar x (begin 2 3 4))
x
```

2

`4`

3

You predicted the output correctly 🎉

The definition binds `x` to the value of the `begin` expression. A `begin` expression evaluates its sub-expressions from left to right and produces the value of the right-most expression. So, `x` is bound to `4`.

4

(Task 3 of 11) What is the result of running this program?

```
(defvar z
  (begin
    (begin
      6
      45)
    (begin
      67)))
z
```

5

`67`

6

You predicted the output correctly 🎉

The value of `(begin (begin 6 45) (begin 67))` is the value of `(begin 67)`, which is `67`.

7

(Task 4 of 11) What is the result of running this program?

```
(defvar x 0)
(defvar y
  (begin
    (set! x (+ x 1))
```

```
    x))
x
y
```

`1 1`

8

You predicted the output correctly 🎉

9

The `begin` first increases `x` by one, then evaluates `x`. So, the value of `y` is `1`.

(Task 5 of 11) What is the result of running this program?

10

```
(defvar x 0)
(defvar y x)
(defvar z
  (begin
    (set! x (+ x 1))
    (set! y 4)
    x))
x
y
z
```

`1 4 1`

11

You predicted the output correctly 🎉

12

At first, `x` is bound to `0`, and `y` is bound to `0`. The `begin` mutates `x` to `1` and `y` to `4`, and returns the value of `x`, which is now `1`.

(Task 6 of 11) The next few questions are going to combine `begin` with `if`.

13

`(if cnd thn els)` evaluates `cnd`. If the evaluation produces a true value, `thn` is evaluated in place of the whole `if` expression. Otherwise, `els` is evaluated.

Giving `if` too few or too many subexpressions is a syntactic error. For example, all of the following programs produce errors.

```
(if (eq? 2 3)
    "Do something")

(if (eq? "apple" "orange")
    "What?!"
    "Okay"
```

```
      "...")
```

Now let's move on to other questions.

(Task 7 of 11) What is the result of running this program?

```
(defvar counter 0)
(deffun (factorial n)
  (if (eq? n 0)
      1
      (set! counter (+ counter 1))
      (* (factorial (- n 1)) n)))
(factorial 2)
```

14

`error`

15

You predicted the output correctly 🎉

16

This is a syntax error! There are 4 sub-expressions in the `if` expression: `(eq? n 0)` and `1` and `(set! ...)` and `(* ...)`. As a result, the language doesn't know which is the condition, which is the then branch, and which is the else branch.

(Task 8 of 11) What is the result of running this program?

17

```
(defvar counter 0)
(deffun (factorial n)
  (if (eq? n 0)
      1
      (begin
        (set! counter (+ counter 1))
        (* (factorial (- n 1)) n))))
(factorial 2)
```

`2`

18

Please briefly explain why you think the answer is `2`.

19

nn

20

You predicted the output correctly 🎉

21

Finally, the program is correct!

(Task 9 of 11) What is the result of running this program? 22

```
(defvar counter 0)
(deffun (factorial n)
  (if (eq? n 0) 1
      (begin
        (set! counter (+ counter 1))
        (* (factorial (- n 1)) n))))
(factorial 2)
```

`2` 23

You predicted the output correctly 🎉🎉🎉 24

This program is still correct! The `then` branch (`1`) is on the same line as the condition (`(eq? n 0)`).

(Task 10 of 11) What did you learn about `begin` from these programs? 25

hh 26

(Task 11 of 11) `begin` computes its sub-expressions in top-to-bottom, left-to-right order, and gives the value of the last expression. 27

Any feedback regarding these statements? Feel free to skip this question. 28

(You skipped the question.) 29

Let's review what we have learned in this tutorial. 30

`begin` computes its sub-expressions in top-to-bottom, left-to-right order, and gives the value of the last expression.

You have finished this tutorial 🎉🎉🎉

Please print the finished tutorial to a PDF file so you can review the content in the future. **Your instructor (if any) might require you to submit the PDF.**

Start time: 1781423332847